

Deterministic Optimization

Large-Scale Optimization

Cutting Stock Problem

Andy Sun

Assistant Professor

Stewart School of Industrial and Systems Engineering

Column Generation for Cutting
Stock Problem

Large-Scale Optimization

Learning Objectives for this lesson

- Apply column generation algorithm to solve cutting stock problem

Cutting Stock Problem

- The Cutting Stock Problem is formulated as
 - $\min \sum_{i=1}^N x_i$
 - s.t. $\sum_{i=1}^N A_i x_i = b$
 - $x \geq 0$
 - where the columns A_i of A are the patterns for how to cut a large roll
 - b is the amount of demand of each size of smaller rolls
 - N is an extremely large number

Column Generation Algorithm

We want to solve a large-scale LP with many variables

1. Pick a subset I of columns $\{A_j\}$ and variables x_j for $j \in I \subseteq \{1, 2, \dots, N\}$
2. Solve (RMP) with a subset of columns I

$$\begin{aligned} (MP) \quad & \min \sum_{j=1}^N x_j \\ & \text{s.t.} \quad \sum_{j=1}^N A_j x_j = b \\ & \quad x_j \geq 0, \quad \forall j = 1, \dots, N. \end{aligned}$$

$$\begin{aligned} (RMP) \quad & \min \sum_{j \in I} x_j \\ & \text{s.t.} \quad \sum_{j \in I} A_j x_j = b \\ & \quad x_j \geq 0, \quad \forall j \in I. \end{aligned}$$

Solving (RMP) is not challenging (and it shouldn't be) because of the small number of columns.

Column Generation Algorithm (Cont.)

3. Pricing Problem: This is challenging!

- Compute $w = \min_{j=1,\dots,N} \{1 - \mathbf{c}_B^T \mathbf{B}^{-1} \mathbf{A}_j\}$
- If $w \geq 0$, then all reduced costs are nonnegative. Terminate column generation.
- If $w < 0$, pick the column j with the minimum reduced cost, add $\mathbf{A}_j$ to (RMP).
- How do we compute ALL the reduced costs without enumerating through all the columns? After all, enumeration is exactly what we want to avoid using column generation!

Coollest Idea for Solving Pricing Problem

- Original pricing problem: $w = \min_{j=1,\dots,N} \{1 - c_B^T B^{-1} A_j\}$
- Equivalent to: $w = 1 - \max_{j=1,\dots,N} \{c_B^T B^{-1} A_j\} = 1 - \max_{j=1,\dots,N} \hat{y}^T A_j$
- Instead of going through all columns and computes $\hat{y}^T A_j$, let's **treat A_j as a variable vector and generate A_j**

$$\begin{aligned} \max_{j=1,\dots,N} \hat{y}^T A_j &= \max \quad \hat{y}^T A \\ \text{s.t.} \quad &A \text{ is a feasible column for the cutting stock problem} \\ &= \max \quad \sum_{i=1}^m \hat{y}_i a_i \end{aligned} \tag{1}$$

$$\begin{aligned} \text{s.t.} \quad &\sum_{i=1}^m w_i a_i \leq W \end{aligned} \tag{2}$$

Knapsack Constraint

$$a_i \geq 0, \text{ integer.} \tag{3}$$

Special Structure of Cutting Stock Problem

- What makes column generation work for cutting stock problem?
 - Special structures of cutting stock problem
- What are special structures of cutting stock problem?
 - All cost coefficients are 1
 - Columns of the constraint matrix A have very special structure – they can be described by a knapsack constraint
 - Solving knapsack problem, although integer program, is quite easy

Summary

- We learned:
 - How to apply Column Generation Algorithm to cutting stock problem
 - Special structures of cutting stock problem